YOLO With Caution: An Implementation of ε-Constraints to Mitigate Algorithmic Bias in YOLO-based Pedestrian Detection Systems

Kaylee Scanlin
Department of Computer Science and Information Technology, Lone Star College, USA
kscanlin@my.lonestar.edu

Dr. Raghu Bollapragada
Operations Research and Industrial Engineering, The University of Texas at Austin, USA
raghu.bollapragada@utexas.edu

Dr. Cem Karamanli
Operations Research and Industrial Engineering, The University of Texas at Austin, USA
cem.karamanli@utexas.edu

Abstract – The YOLO (You Only Look Once) algorithm represents a state-of-the-art method in computer vision, widely utilized in autonomous vehicle systems for object detection on the road. However, extant research indicates that the algorithm exhibits challenges in accurately detecting pedestrians underrepresented in the training data, including cyclists, children, and pedestrians of color.

This project aims to implement epsilon (ε) constraints during the training of the YOLO v1 model to mitigate biases inherent in the training dataset with a focus in pedestrian data. The study will compare the model's error and bias metrics with and without the ε-constraints, examining their impact on the false negative rate, true positive rate, accuracy, and precision.

Keywords – ***Machine Learning, Algorithmic Bias, YOLO, ε-Constraint***

I. INTRODUCTION

Autonomous vehicles (AVs) have demonstrated significant potential in mitigating human error in traffic accidents. These vehicles are equipped with advanced sensors and cameras that enable them to detect objects from greater distances, facilitate rapid emergency braking, and provide lane-keeping assistance. However, as the technology underpinning self-driving cars is still in its nascent stages, numerous risks persist.

Non-security-related risks associated with AVs predominantly stem from software and computer vision challenges. The primary algorithms for object detection in AV systems, such as R-CNN and YOLO-based methods, are influenced by the biases present in the datasets upon which they are trained. Furthermore, the complexity of real-world driving conditions, characterized by high speeds, rapidly changing decisions, and adverse weather and lighting conditions further complicates the performance of these systems.

One problem pertaining to algorithmic bias occurs when there is bias present within the dataset. When models are trained on datasets that are under-representative of certain demographic groups, image detection models recognize them disproportionately. The mitigation of biases within these models is typically counteracted within the convolutional neural networks trained on these models by applying constraints on their functions.

II. PRIOR WORK

This study builds upon the YOLO (You Only Look Once) algorithm, originally developed by Redmon et al. [1], by incorporating epsilon constraints to enhance its robustness. The YOLO v1 algorithm marks a significant innovation in the domain of object detection, offering a streamlined method whereby both bounding boxes and class probabilities are predicted directly from full images through a single forward pass of the neural network.

In contrast to other SOTA algorithms for image detection, YOLO v1 conceptualizes object detection as a singular regression problem, converting raw image pixels directly into bounding box coordinates and corresponding class probabilities. The algorithm partitions the image into an S×S grid, assigning to each grid cell the task of predicting bounding boxes and associated confidence scores, along with class probabilities for these boxes.

Architecturally, the YOLO v1 model is structured as a convolutional neural network (CNN) composed of 24 convolutional layers succeeded by 2 fully connected (dense) layers. The model includes layers of 1x1 reduction followed by 3x3 convolutional layers. Upon processing, the image is inputted into the CNN as a complete batch during a single computational pass, subsequently outputting a tensor of dimensions S x S x (B x 5 + C). Here, S x S denotes the dimensions of the grid cells, B represents the number of bounding boxes each cell is capable of predicting (encompassing five parameters: center x-coordinate, center y-coordinate, width, height, and confidence score), and C indicates the number of possible class probabilities.

III. APPROACH

To assess the effectiveness of bias mitigation strategies in object detection systems, we initially constructed a specialized dataset tailored to represent a single demographic group under specific environmental conditions. This dataset is composed of 748 augmented and non-augmented images of a dark-skinned female pedestrian navigating the University of Texas at Austin campus during nighttime hours. Each image was annotated in the YOLO v1 format utilizing Roboflow, ensuring a consistent framework for model training and evaluation.

Employing this dataset, we designed and trained a convolutional neural network (CNN) based on the architecture specified in the seminal YOLO v1 paper (Redmon et al., 2015) using the Keras deep learning library. The model's performance was subsequently evaluated against various raw subsets extracted from the KAIST multispectral pedestrian dataset [3], differentiated by diverse lighting conditions and backgrounds to test generalizability.

Evaluation Sets

The evaluation was structured across several datasets, characterized as follows:

- **Set 00**: Multi-ethnic and gendered, daytime, campus setting; 17,498 frames.
- **Set 01**: Multi-ethnic and gendered, daytime, roadside; 8,035 frames.
- **Set 02**: Multi-ethnic and gendered, daytime, downtown; 7,866 frames.
- **Set 03**: Multi-ethnic and gendered, nighttime, campus; 6,668 frames.
- **Set 04**: Multi-ethnic and gendered, nighttime, roadside; 7,200 frames.
- **Set 05**: Multi-ethnic and gendered, nighttime, downtown; 2,920 frames.
- **Set 06** (curated for demonstrating experimental results): Multi-ethnic and gendered, variable lighting conditions, assorted environments (campus, road, and downtown); 1,036 frames.

Set 06 was hand-composed from examples from the prior subsets for demonstrative purposes to serve as an average of each set. It was composed of 171-173 images from each subset 00-05.

Model Enhancement and Evaluation

To enhance the model’s predictive accuracy, an epsilon constraint [2] was incorporated during the preprocessing phase to adjust the contrast levels of the images and the dimensions of the bounding boxes. The model's predictive performance was quantified through the Keras library function model.evaluate() for each dataset, calculating and providing an average for the defined loss function in section IV.

A customized loss function, derived from the original YOLO v1 formulation, was employed to quantitatively assess the discrepancies between the predicted and actual bounding box coordinates for each evaluated subset, outlined in Section IV.

A. Creating the Dataset

To construct our intentionally-biased dataset, we collected images featuring a singular dark-skinned, female pedestrian traversing the University of Texas at Austin's campus during evening hours. This dataset was designed to train a model specifically to identify pedestrians of a particular demographic under consistent lighting conditions. A total of 187 original images were captured using an iPhone 12s camera. These images were subsequently annotated and augmented using the Roboflow platform. Augmentation techniques included rotations of 90 and 180 degrees, along with adjustments to contrast and color saturation. Through these methods, we expanded the dataset to comprise 748 images in total, thereby enhancing the diversity of data available for training the model without straying from the intended demographic focus.

Figure 1. Example Data from Custom Set.

B. Cleaning and Normalizing the Data

Each image in the dataset underwent a systematic inspection within a loop to identify and exclude corrupted files. This process checked for inappropriate file extensions and data integrity issues. Subsequently, all valid images were uniformly resized to a resolution of 448x448 pixels utilizing the OpenCV library. Following resizing, each image was converted into a matrix representation, wherein the RGB values of each pixel were normalized to a range between 0 and 1 to facilitate subsequent processing.

C. Development and Training of the Convolutional Neural Network (CNN)

The neural network was developed utilizing the Keras API for TensorFlow. The architecture begins with an input layer designed to process images of dimensions 448 x 448, with each pixel containing three color channels (Red, Green, Blue), thus forming an input layer of 448 x 448 x 3. The model comprises 32 hidden layers, which do not include the input and output layers. These hidden layers consist of 13 convolutional layers (Conv2D), which are utilized to extract features from the image using various kernels. Accompanying these are 13 batch normalization layers, which serve to normalize the activations from the preceding layers, enhancing the network's stability and performance.

Dimensionality reduction is achieved through 5 MaxPooling layers, which systematically reduce the spatial dimensions of the output from the convolutional layers. The architecture also includes a single dense (fully-connected) layer, which is tasked with performing the final classification. The LeakyReLU function is employed as the activation mechanism across the network, providing a means to introduce non-linearity and mitigate the issue of vanishing gradients:

$$leakyrelu(z) = \begin{cases} 0.01z \; for \; z < 0 \\ z \; for \; z \geq 0 \end{cases}$$

Figure 2. LeakyReLU function for activation.

For model training, the original loss function of the YOLO algorithm was employed and subsequently modified to better quantify discrepancies between the actual bounding boxes of each image in the KAIST multispectral subsets and those predicted by our models, as detailed in Section IV.

The Adam optimization algorithm was selected for its efficiency in handling sparse gradients and adaptive learning rate capabilities. The model underwent training over 100 epochs with a batch size of 32. During each training cycle, the model classified the batch, computed the loss, calculated the gradients, updated the optimizer based on these gradients, and then reset the gradients in preparation for the next batch.

Each subset of the KAIST Multispectral dataset was then tested on this model, and the average loss between the calculated bounding box coordinates and the true bounding boxes provided in the dataset was calculated and displayed. Additionally, the loss was calculated once more after the epsilon constraints were added during the pre and post-processing stage using the model.evaluate() function in Keras.

IV. THE LOSS FUNCTION

The loss function used to measure the accuracy of our model is taken from the loss function in the original paper by Redmon et al [1]. It is composed of the coordinates for the bounding boxes generated by the trained YOLO model subtracted from the true values of the bounding boxes present in the original KAIST Multispectral Pedestrian subsets:

$$\lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{\theta} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{\theta} \mathbb{1}_{ij}^{obj} \left[\left(\sqrt{w}_i - \sqrt{\widehat{w_i}} \right)^2 + \left(\sqrt{h}_i - \sqrt{\widehat{h_i}} \right)^2 \right]$$

Figure 3. Modified Loss Function for YOLO with Epsilon Constraint.

This is a squared error function denoting the subtraction of the x and y center coordinates of the bounding boxes, the width of the bounding boxes, and the height of the bounding boxes. λ_{coord} is an arbitrary scaling factor to maximize the importance of localization errors, $\sum_{i=0}^{S^1} \sum_{j=0}^{\theta}$ is the summation of values for each bounding box, $\mathbb{1}_{ij}^{obj}$ is the indicator function to denote that only grid squares with an object detected in them are to be considered, and the total loss

is a summation of the squared error function of the prior variables.

V. NETWORK ARCHITECTURE

The architecture of the neural network is as follows:

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 448, 448, 3)	0
conv2d (Conv2D)	(None, 224, 224, 64)	9,472
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256
leaky_re_lu (LeakyReLU)	(None, 224, 224, 64)	0
max_pooling2d (MaxPooling2D)	(None, 112, 112, 64)	0
conv2d_1 (Conv2D)	(None, 112, 112, 192)	110,784
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 192)	768
leaky_re_lu_1 (LeakyReLU)	(None, 112, 112, 192)	0
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 192)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	221,312
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 128)	512
leaky_re_lu_2 (LeakyReLU)	(None, 56, 56, 128)	0
conv2d_3 (Conv2D)	(None, 56, 56, 256)	295,168
batch_normalization_3 (BatchNormalization)	(None, 56, 56, 256)	1,024
leaky_re_lu_3 (LeakyReLU)	(None, 56, 56, 256)	0
conv2d_4 (Conv2D)	(None, 56, 56, 256)	590,080
batch_normalization_4 (BatchNormalization)	(None, 56, 56, 256)	1,024
leaky_re_lu_4 (LeakyReLU)	(None, 56, 56, 256)	0
conv2d_5 (Conv2D)	(None, 56, 56, 512)	1,180,160
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 512)	2,048
leaky_re_lu_5 (LeakyReLU)	(None, 56, 56, 512)	0
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 512)	0
conv2d_6 (Conv2D)	(None, 28, 28, 512)	2,359,808
batch_normalization_6 (BatchNormalization)	(None, 28, 28, 512)	2,048
leaky_re_lu_6 (LeakyReLU)	(None, 28, 28, 512)	0
conv2d_7 (Conv2D)	(None, 28, 28, 512)	2,359,808
batch_normalization_7 (BatchNormalization)	(None, 28, 28, 512)	2,048
leaky_re_lu_7 (LeakyReLU)	(None, 28, 28, 512)	0
conv2d_8 (Conv2D)	(None, 28, 28, 512)	2,359,808
batch_normalization_8 (BatchNormalization)	(None, 28, 28, 512)	2,048
leaky_re_lu_8 (LeakyReLU)	(None, 28, 28, 512)	0
conv2d_9 (Conv2D)	(None, 28, 28, 512)	2,359,808
batch_normalization_9 (BatchNormalization)	(None, 28, 28, 512)	2,048
leaky_re_lu_9 (LeakyReLU)	(None, 28, 28, 512)	0
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 512)	0
conv2d_10 (Conv2D)	(None, 14, 14, 1024)	4,719,616
batch_normalization_10 (BatchNormalization)	(None, 14, 14, 1024)	4,096
leaky_re_lu_10 (LeakyReLU)	(None, 14, 14, 1024)	0
conv2d_11 (Conv2D)	(None, 14, 14, 1024)	9,438,208
batch_normalization_11 (BatchNormalization)	(None, 14, 14, 1024)	4,096
leaky_re_lu_11 (LeakyReLU)	(None, 14, 14, 1024)	0
conv2d_12 (Conv2D)	(None, 7, 7, 1024)	9,438,208
batch_normalization_12 (BatchNormalization)	(None, 7, 7, 1024)	4,096
leaky_re_lu_12 (LeakyReLU)	(None, 7, 7, 1024)	0
conv2d_13 (Conv2D)	(None, 7, 7, 1024)	9,438,208
batch_normalization_13 (BatchNormalization)	(None, 7, 7, 1024)	4,096
leaky_re_lu_13 (LeakyReLU)	(None, 7, 7, 1024)	0
conv2d_14 (Conv2D)	(None, 7, 7, 1024)	9,438,208
batch_normalization_14 (BatchNormalization)	(None, 7, 7, 1024)	4,096
leaky_re_lu_14 (LeakyReLU)	(None, 7, 7, 1024)	0
flatten (Flatten)	(None, 50176)	0
dense (Dense)	(None, 4096)	205,524,992
leaky_re_lu_15 (LeakyReLU)	(None, 4096)	0
dense_1 (Dense)	(None, 294)	1,204,518
reshape (Reshape)	(None, 7, 7, 6)	0

VI. RESULTS

Results are graphed and compiled into tabular form below for each set.

Set	Pre-Epsilon Constraint	Post-Epsilon Constraint
00	206.2713	216.3524
01	204.9364	218.3677
02	203.2714	220.4931
03	203.1776	222.5655
04	196.9213	226.5779
05	196.0473	225.0632
06	193.2714	230.0819

Figure 4. Average loss table over 100 iterations, pre and post-epsilon constraint calculated with model.evaluate().

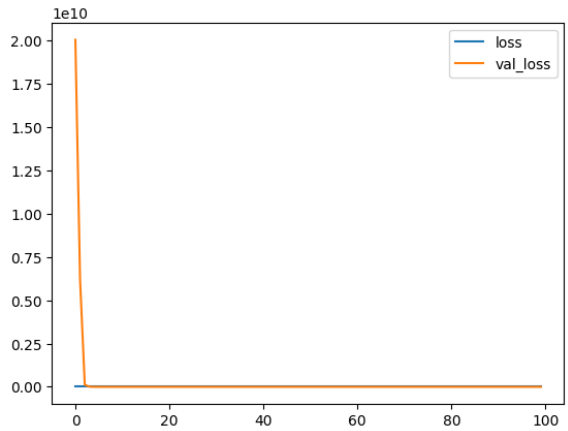

Figure 5. Average loss function of the training data whilst training the model.

VII. CONCLUSION

It is worth noting that the average loss between all images is still incredibly high due to the extremely biased and selective nature of our dataset. However, in our analysis, the minimum improvement in the loss function was observed in subset 00 at approximately 4.89%. This was anticipated, given that Set 00 is the largest subset, characterized by the most varied lighting and environmental conditions that significantly diverge from those in our biased training dataset. The maximum improvement was recorded in subset 05, where the loss function improvement reached approximately 11.54%. Upon further analysis of the data, a clear correlation between the similarity of the training and testing datasets and the effectiveness of the model in approximating the true bounding box coordinates, with subsets most similar to the training data (Sets 03, 04, and 05) exhibiting the greatest improvements.

This minor success demonstrates that it is not only possible to mitigate algorithmic bias within the network itself, but at the pre-processing and post-processing stages as well. In accounting for this bias, it is worth noting that the ability to mitigate bias within these stages indicates that it is possible to unknowingly develop bias within them as well. Research into algorithmic bias within the computer vision field has extensive applications, and demonstrating that it is possible to find and counteract bias at any point in the algorithm indicates that further work in the field of algorithmic bias should take into consideration how it develops in the pre-processing and post-processing stages as well.

VIII. FUTURE WORK

Future work could involve expanding the usage of epsilon constraints into the other commonly-used computer vision algorithms, YOLO v3, v5, and v8, as well as implement further epsilon constraints into the post-processing stage to maintain demographic parity and ensure that the sensitivity of the model does not exceed a small arbitrary percentage for any demographic group.

IX. ACKNOWLEDGEMENTS

We gratefully acknowledge the National Science Foundation (NSF) which supported this work through NSF Research Experience for Undergraduates (REU) Site: CI Research 4 Social Change REU, Award # 2150390. This work was supported by the Texas Advanced Computing Center (TACC), and the REU program was organized by Rosalia Gomez. Additionally, further gratitude is extended to Dr. Raghu Bollapragada and Dr. Cem Karamanli, who served as REU mentors for this experience.

X. REFERENCES

[1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://doi.org/10.1109/cvpr.2016.91

[2] Curtis, F. E., Liu, S., & Robinson, D. P. (2023). Fair machine learning through constrained stochastic optimization and an epsilon-constraint method. *Optimization Letters*. https://doi.org/10.1007/s11590-023-02024-6

[3] Hwang, S., Park, J., Kim, N., Choi, Y., & Kweon, I. S. (2015). Multispectral pedestrian detection: Benchmark dataset and baselines. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.